

Fundamentos de Sistemas Robóticos com ROS 2

Etapa 7

Professor Mario Pego

mario.pegos@prof.infnet.edu.br



Cronograma



Fundamentos de Sistemas Robóticos com ROS 2

<i>Etapas</i>	<i>Segunda-feira/Quarta-feira</i>
Etapa 1	20/07/26 e 22/07/26
Etapa 2/Leitura TP1	27/07/26 e 29/07/26
Etapa 3	03/08/26 e 05/08/26
Etapa 4/Entrega do TP1/Leitura TP2	10/08/26 e 12/08/2026
Etapa 5	17/08/26 e 19/08/26
Etapa 6/Entrega TP2/Leitura do TP3	24/08/26 e 26/08/26
Etapa 7	31/08/26 e 02/09/26
Etapa 8/Entrega TP3	07/09/26 (FERIADO) e 09/09/26
Etapa 9/Leitura do AT	14/09/26 e 16/09/26
Entrega Assessment	21/09/26 e 23/09/26
Reentrega	28/09/26 e 30/09/26

Parametrizar nós ROS 2 dinamicamente para configuração em tempo de execução

O que vamos ver

1

O que são parâmetros ROS 2

O quarto jeito de um nó receber informação, além de tópico, serviço e ação

2

declare_parameter e get_parameter

Como declarar um valor configurável e lê-lo dentro do nó

3

O subsistema de monitoramento

Grafo de computação do robô hospitalar do AT

4

monitor_transporte.py

O nó completo do AT Exercício 2

O que já sabemos combinar

Das Etapas 5 e 6 — tópicos, serviço, ação, tudo no mesmo nó

- controlador.py (Etapa 5): serviço + estado interno + Lock
- patrulha_server.py (Etapa 6): ação com feedback contínuo
- coordenador_vigilancia.py (Etapa 6): tópico + serviço reagindo em conjunto
- Hoje entramos no AT: o mesmo tipo de nó, mas agora com um valor que muda SEM recompilar — o parâmetro

Parâmetros: configuração sem recompilar

O quarto mecanismo do ROS 2, ao lado de tópico, serviço e ação

- Até agora, valores como 'temperatura_limite = 28.0' estariam escritos direto no código — mudar exige editar, recompilar e reiniciar
- Um parâmetro ROS 2 é um valor nomeado, associado a UM nó, que pode ser lido e alterado em tempo de execução
- Casos de uso reais: limiares de alarme, taxas de publicação, nomes de tópicos, caminhos de arquivo — qualquer coisa que varie entre ambientes (simulação vs robô real, por exemplo)
- O AT pede exatamente isso: um limiar de temperatura configurável sem reiniciar o nó

declare_parameter e get_parameter

Todo parâmetro precisa ser declarado antes de ser lido

```
self.declare_parameter(  
    'temperatura_limite', 28.0)  
  
limite = self.get_parameter(  
    'temperatura_limite').value
```

Por que declarar primeiro

declare_parameter registra o nome, o tipo (inferido do valor default, aqui float) e um valor inicial. Tentar ler um parâmetro não declarado lança exceção — é uma proteção contra erros de digitação no nome.

Três formas de definir o valor inicial

Todas alimentam o mesmo `declare_parameter` — a diferença é só a origem

Forma	Quando usar	Exemplo
Default no código	Valor razoável para o caso comum	<code>declare_parameter('x', 28.0)</code>
<code>--ros-args -p</code> na linha de comando	Ajuste pontual, um único nó	<code>ros2 run pkg no --ros-args -p x:=25.0</code>
Arquivo YAML	Muitos parâmetros, reuso em Launch files	<code>--ros-args --params-file config.yaml</code>

Grafo de computação: subsistema de monitoramento

Um único nó concentra as quatro interfaces pedidas pelo AT

Interface	Direção	Mensagem / tipo
/temperatura_ambiente	publica a cada 1s	sensor_msgs/msg/Temperature
/robo_ocupado	publica a cada 1s	std_msgs/msg/Bool
/confirmar_entrega	expõe (serviço)	std_srvs/srv/Trigger
/alertas	publica (condicional)	std_msgs/msg/String
temperatura_limite	parâmetro (float)	default 28.0

Anatomia do construtor

Declarar o parâmetro é só mais uma linha no `__init__`

```
def __init__(self):  
    super().__init__('monitor_transporte')  
    self.declare_parameter(  
        'temperatura_limite', 28.0)  
    self.ocupado = True  
    # ... publishers e serviço aqui
```

Nome do parâmetro = string

'temperatura_limite' é só uma string — o ROS 2 não sabe nada sobre o que ela significa. É responsabilidade do código ler esse valor no lugar certo (aqui, dentro do `tick()`).

O serviço /confirmar_entrega (Trigger)

std_srvs/srv/Trigger não tem campos de request

```
$ ros2 interface show std_srvs/srv/Trigger
---
bool success
string message

def confirmar_entrega_callback(
    self, request, response):
    self.ocupado = False
    response.success = True
    return response
```

Request vazio

Trigger é o serviço 'mais simples possível': nada no request (por isso 'ros2 service call' usa '{}'), só success/message na resposta. Ideal para ações do tipo botão — sem parâmetros de entrada.

Onde ler o parâmetro importa

get_parameter dentro do tick(), não guardado em self no __init__

```
def tick(self):  
    limite = self.get_parameter(  
        'temperatura_limite').value  
    if temperatura > limite:  
        # ... publica alerta
```

Por que não cachear

Se limite fosse lido só uma vez no __init__ e guardado em self.limite, um 'ros2 param set' depois não teria efeito nenhum — o valor ficaria congelado. Ler get_parameter a cada tick é o que permite reconfigurar sem reiniciar (Competência 2).

monitor_transporte.py — parte 1 de 4

Imports e início do construtor: parâmetro e primeiro publisher

salvar em `comunicacao_ros2/comunicacao_ros2/monitor_transporte.py`

```
import random
import rclpy
from rclpy.node import Node
from rclpy.executors import MultiThreadedExecutor
from sensor_msgs.msg import Temperature
from std_msgs.msg import Bool, String
from std_srvs.srv import Trigger
class MonitorTransporte(Node):
    def __init__(self):
        super().__init__('monitor_transporte')
        self.declare_parameter('temperatura_limite', 28.0)
        self.ocupado = True
        self.pub_temp = self.create_publisher(
            Temperature, '/temperatura_ambiente', 10)
```

monitor_transporte.py — parte 2 de 4

Fim do construtor e o callback do serviço

salvar em `comunicacao_ros2/comunicacao_ros2/monitor_transporte.py`

```
self.pub_ocupado = self.create_publisher(
    Bool, '/robo_ocupado', 10)
self.pub_alertas = self.create_publisher(
    String, '/alertas', 10)
self.srv_entrega = self.create_service(
    Trigger, '/confirmar_entrega',
    self.confirmar_entrega_callback)
self.create_timer(1.0, self.tick)
self.get_logger().info('nó iniciado')

def confirmar_entrega_callback(self, request, response):
    self.ocupado = False
    alerta = String()
    alerta.data = 'Entrega confirmada'
    self.pub_alertas.publish(alerta)
    response.success = True
    response.message = 'Entrega confirmada'
    return response
```

monitor_transporte.py — parte 3 de 4

tick(): publica temperatura, ocupação e checa o limiar

salvar em `comunicacao_ros2/comunicacao_ros2/monitor_transporte.py`

```
def tick(self):
    temp_msg = Temperature()
    temp_msg.temperature = round(random.uniform(18.0, 30.0), 2)
    self.pub_temp.publish(temp_msg)
    ocupado_msg = Bool()
    ocupado_msg.data = self.ocupado
    self.pub_ocupado.publish(ocupado_msg)
    limite = self.get_parameter('temperatura_limite').value
    if temp_msg.temperature > limite:
        alerta = String()
        alerta.data = 'TEMPERATURA FORA DO PADRAO'
        self.pub_alertas.publish(alerta)
```

monitor_transporte.py — parte 4 de 4

main() com MultiThreadedExecutor

salvar em `comunicacao_ros2/comunicacao_ros2/monitor_transporte.py`

```
def main(args=None):
    rclpy.init(args=args)
    node = MonitorTransporte()
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    try:
        executor.spin()
    finally:
        executor.shutdown()
        node.destroy_node()
        rclpy.shutdown()
```

Registrando o `entry_point`

`monitor_transporte` precisa entrar no `setup.py`

salvar em `comunicacao_ros2/setup.py`

```
entry_points={
    'console_scripts': [
        # ... controlador, estado_robo,
        # ... patrulha_server, patrulha_client,
        # ... coordenador_vigilancia
        'monitor_transporte = '
        'comunicacao_ros2.monitor_transporte:main',
    ],
},
```

Décimo executável

Primeiro nó do pacote `comunicacao_ros2` que usa parâmetro — os anteriores (TP2/TP3) não precisavam de valores configuráveis em tempo de execução.

Compilando e subindo o monitor_transporte

colcon build + ros2 run — parâmetro com default 28.0

```
$ cd ~/tp1_ws  
$ colcon build --packages-select comunicacao_ros2  
$ source install/setup.bash  
$ ros2 run comunicacao_ros2 monitor_transporte
```

```
[INFO] [monitor_transporte]: nó iniciado
```

Deixe rodando

Mantenha este terminal aberto — os próximos passos usam outros terminais para chamar o serviço e inspecionar o parâmetro.

Conferindo a publicação de temperatura

ros2 topic echo — valores aleatórios entre 18.0 e 30.0

```
$ ros2 topic echo /temperatura_ambiente
temperature: 24.75
---
temperature: 26.10
---
```

Aleatório, não determinístico

Cada execução gera valores diferentes — é esperado. O que importa para a validação é o comportamento acima do limiar, não um valor específico.

Testando o serviço /confirmar_entrega

Terminal 1: chama o serviço. Terminal 2: observa /alertas

Terminal 1 — ros2 service call

```
$ ros2 service call \  
  /confirmar_entrega \  
  std_srvs/srv/Trigger "{}"
```

```
response:  
success=True
```

Terminal 2 — ros2 topic echo

```
$ ros2 topic echo /alertas  
  
data: Entrega confirmada  
---
```

/robo_ocupado também muda

Depois da chamada, 'ros2 topic echo /robo_ocupado' deve mostrar 'data: false' — confira nos próprios prints do relatório.

Conferindo o parâmetro declarado

ros2 param get — critério de validação do AT

```
$ ros2 param get /monitor_transporte \
    temperatura_limite
```

```
Double value is: 28.0
```

Ainda o default

28.0 é o valor declarado no código — na Competência 2, vamos mudar isso em tempo real com 'ros2 param set', sem reiniciar o nó.

Parâmetros sem armadilhas comuns

- Sempre declare `_parameter` antes de `get_parameter` — a ordem importa, e o tipo do default define o tipo aceito depois
- Leia o parâmetro no ponto de uso (dentro do callback/timer), nunca só uma vez no `__init__`, se ele precisa ser reconfigurável
- Nomes de parâmetro em `snake_case`, como tópicos e serviços — consistência facilita ler o 'ros2 param list' de um nó grande
- Um valor default sensato no código é a rede de segurança — o nó deve funcionar mesmo se ninguém passar `--ros-args -p`

Os quatro mecanismos do ROS 2, lado a lado

Mecanismo	Quando usar	Exemplo nesta disciplina
Tópico	Dado contínuo, sem resposta	estado_robo.py
Serviço	Pergunta pontual, síncrona	controlador.py
Ação	Tarefa longa, com progresso	patrulha_server.py
Parâmetro	Configuração sem recompilar	monitor_transporte.py

OK

`declare_parameter / get_parameter`

declarar e ler um parâmetro com valor default

OK

`monitor_transporte.py completo`

tópicos, serviço Trigger e parâmetro, todos funcionando

OK

`ros2 param get`

inspecionar o valor atual de um parâmetro

OK

`Ler get_parameter no ponto de uso`

obrigatório para reconfiguração dinâmica

Próximo passo

Com `monitor_transporte.py` pronto e o parâmetro `temperatura_limite` declarado, falta demonstrar a parte que dá nome a esta etapa: mudar esse valor com o nó já rodando, sem reiniciar. Isso é a Competência 2 — configuração dinâmica em tempo de execução.

Configurar nós dinamicamente em tempo de execução

O que vamos ver

1

Por que 'sem recompilar' importa

O cenário real que motiva parâmetros dinâmicos

2

A CLI de parâmetros

ros2 param list, get, set e dump

3

Reconfigurando o monitor_transporte.py ao vivo

Mudando o limiar com o nó já rodando

4

--ros-args -p e arquivo YAML

As outras duas formas de definir um parâmetro

O que já construímos

Da Competência 1 desta etapa

- `monitor_transporte.py` já publica `/temperatura_ambiente`, `/robo_ocupado` e `/alertas`
- O parâmetro `temperatura_limite` já está declarado, com default 28.0
- O `tick()` já lê `get_parameter` a cada ciclo — não guarda o valor em cache
- Hoje não escrevemos nenhuma linha de código novo: só vamos operar o nó que já existe, de um jeito diferente

Por que 'sem recompilar' importa de verdade

O cenário que a validação do AT está simulando

- Um robô hospitalar real já está em operação, monitorando pacientes — não dá para parar tudo, editar código e reiniciar por causa de um limiar mal calibrado
- Times de operação (não de desenvolvimento) frequentemente precisam ajustar limiares, sem acesso ao código-fonte nem permissão para recompilar
- É exatamente esse o valor de `declare_parameter` + leitura no ponto de uso: o comportamento muda, o binário não
- A validação do AT testa isso de forma literal: mudar o valor, confirmar com `get`, e vê o comportamento mudar — tudo com o processo já em execução

A CLI de parâmetros do ROS 2

Quatro subcomandos cobrem praticamente tudo

Comando	O que faz
<code>ros2 param list</code>	Lista todos os parâmetros declarados por um nó
<code>ros2 param get <nó> <param></code>	Lê o valor atual de um parâmetro
<code>ros2 param set <nó> <param> <valor></code>	Altera o valor em tempo real, nó já rodando
<code>ros2 param dump <nó></code>	Exporta todos os parâmetros do nó em formato YAML

O detalhe que faz 'ros2 param set' funcionar

```
# RUIM — lido uma vez, nunca mais muda
def __init__(self):
    self.limite = self.get_parameter(
        'temperatura_limite').value

# BOM — lido de novo a cada tick
def tick(self):
    limite = self.get_parameter(
        'temperatura_limite').value
```

Mesma API, lugar diferente

`get_parameter()` sempre retorna o valor MAIS RECENTE. O bug do 'RUIM' não é a chamada em si, é guardar o resultado em `self` e nunca chamar de novo.

`monitor_transporte.py` já está certo, está no método `tick` que roda a cada 1 segundo. — é só confirmar isso na prática agora.

Listando os parâmetros do nó

ros2 param list — com o monitor_transporte já rodando (Competência 1)

```
$ ros2 param list /monitor_transporte
temperatura_limite
use_sim_time
```

use_sim_time não é nosso

Todo nó rclpy ganha automaticamente esse parâmetro (usado para sincronizar com tempo simulado, relevante a partir da Etapa 9 com Gazebo).

Valor atual, antes da mudança

ros2 param get — ainda o default declarado no código

```
$ ros2 param get /monitor_transporte \
  temperatura_limite
```

```
Double value is: 28.0
```

Ponto de partida

Registre este valor antes de mudar — o relatório do AT pede o antes e o depois lado a lado.

Mudando o limiar com o nó rodando

ros2 param set — sintaxe exata do AT

```
$ ros2 param set /monitor_transporte temperatura_limite 25.0
```

```
Set parameter successful
```

O nó nunca parou

Nenhum colcon build, nenhum ros2 run de novo — o processo do monitor_transporte.py continua o mesmo desde a Competência 1.

Confirmando a mudança: valor e comportamento

Terminal 1: novo valor. Terminal 2: alerta no novo limiar

Terminal 1 — ros2 param get

```
$ ros2 param get \  
  /monitor_transporte \  
  temperatura_limite
```

```
Double value is: 25.0
```

Terminal 2 — ros2 topic echo /alertas

```
$ ros2 topic echo /alertas  
  
data: TEMPERATURA FORA DO PADRAO  
---
```

O critério do AT

Antes do set, só valores acima de 28.0 disparavam alerta. Depois, 25.0 já é suficiente — e isso aconteceu sem reiniciar o nó. É exatamente essa comparação que o relatório precisa evidenciar.

Definindo o valor já na inicialização

--ros-args -p — alternativa a mudar depois com param set

```
$ ros2 run comunicacao_ros2 \  
  monitor_transporte \  
  --ros-args -p temperatura_limite:=25.0
```

```
[INFO] [monitor_transporte]: nó iniciado
```

Diferença para param set

Aqui o valor já nasce em 25.0 — não muda um nó em execução, define o valor ANTES do __init__ rodar. Útil quando você já sabe o valor certo de antemão.

Arquivo YAML

Útil quando há muitos parâmetros, ou para reuso em Launch files

```
# config/monitor_transporte.yaml
monitor_transporte:
  ros__parameters:
    temperatura_limite: 26.0
```

Como usar

`ros2 run comunicacao_ros2`
`monitor_transporte --ros-args --params-file`
`config/monitor_transporte.yaml`. O nome do
nó no YAML ('monitor_transporte:') precisa
bater exatamente com o nome real do nó —
é a causa mais comum de 'o YAML não fez
nada'.

```
ros2 run comunicacao_ros2 monitor_transporte --ros-args --params-file ~tp1_ws/src/comunicacao_ros2/config/monitor_transporte.yaml
```

Reconfiguração em produção

- Sempre confira com 'ros2 param get' depois de um 'ros2 param set' — o comando não avisa se o nome do parâmetro estiver errado silenciosamente aceito
- Prefira YAML a vários '-p' na linha de comando quando o nó tiver mais de 2-3 parâmetros — mais fácil de versionar e revisar
- Nem todo parâmetro deveria ser dinâmico: valores estruturais (como o nome de um tópico) geralmente exigem reiniciar o nó de qualquer forma
- Para reagir IMEDIATAMENTE a uma mudança de parâmetro (em vez de só no próximo tick), o rclpy oferece `add_on_set_parameters_callback` — fora do escopo do AT, mas bom saber que existe

As três formas de definir um parâmetro, lado a lado

Forma	Quando o valor é fixado	Precisa reiniciar o nó?
Default no código	Em tempo de compilação	Sim, para mudar
<code>--ros-args -p</code>	Na inicialização do processo	Sim (é um novo processo)
<code>ros2 param set</code>	A qualquer momento, nó rodando	Não

OK

`ros2 param list / get / set`

inspecionar e alterar parâmetros de um nó em execução

OK

Reconfiguração sem reiniciar

confirmar o antes/depois de temperatura_limite com prints

OK

`--ros-args -p vs YAML vs param
set`

explicar quando cada forma faz sentido

OK

AT Exercício 2 e metade do 3

monitor_transporte.py e a reconfiguração dinâmica cobertos

Próximo passo

Com o subsistema de monitoramento parametrizável e reconfigurável, falta a segunda metade do AT

Exercício 3: descrever o hardware do robo_hospitalar em URDF. Na Etapa 8, vamos construir essa descrição XML — base, rodas e joints — preparando o terreno para a simulação no Gazebo da Etapa 9.